
6. Implementing the Interaction Diagrams

The interaction diagrams in UML are the sequence and collaboration diagram. They depict the relationships between the objects in a system and objects work together to perform a particular task. It **defines a functionality of the system** by demonstrating **a set of chronological interactions between objects**. It is basically used to demonstrate how objects interrelate to **perform the behaviour of a specific use case**.

6.1 Creating Methods from the Collaboration Diagram

A collaboration diagram carries the similar type of information as a sequence diagram; it focuses on the *object structure instead of the chronology of messages passing between them*. The collaboration diagram is also called a **communication diagram** and is used to **represent the object's architecture in the system**. Collaboration diagrams are appropriate for **analyzing use case diagrams**. This diagram can be used to show the **dynamic behaviour of a specific use case** and describes the **role of each object**.

The key purpose of creating the collaboration diagram is

- to illustrate the **relationship between objects** and works together to perform a particular task. This task is difficult to regulate from a sequence diagram.
- to support developer in controlling the **accuracy of the static model**, such as **class diagrams**.

In an application system, there are one or more collaboration diagrams to find all the eventuality of a complex behaviour. The collaboration diagram contains the elements such as **actors, objects** and their **communication links**, and **messages**.

- **Each actor** performs a significant role as it **invokes the interaction and has a name**.
- The **link** behaves as a **connector between the objects and actors**. It depicts a relationship between the objects through which the messages are sent.
- The **messages** are **exchanged between objects in an order** which is represented by sequence numbers. The messages are sent from the sender to the recipient, and the direction must be traversable in that particular direction.

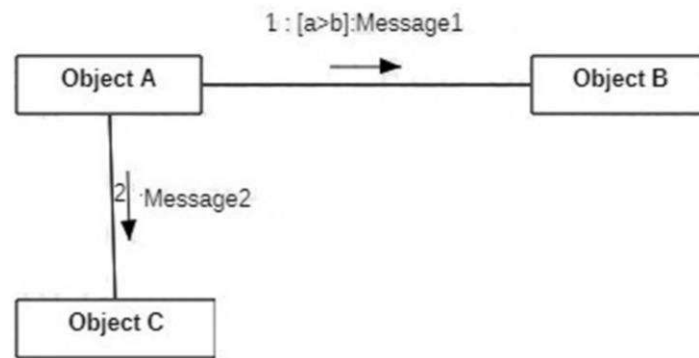


Figure 1.37: Collaboration Diagram showing If-else condition

The implementation of the above diagram, Figure 1.37 is as follows:

```

if (a>b) {
    ObjectB.Message1();
}
else {
    ObjectC.Message2();
}
  
```

6.2 Implementing the Collaboration Diagram Example

Let us consider a collaboration diagram for ATM transactions. For this example, first, create a Use Case diagram and then draw a collaboration diagram. For building a **Use Case diagram**, use the below flow of controls:

- Customer can insert ATM card into the card reader of the ATM console.
- The card reader reads and verifies the card.
- ATM console displays a message and prompts the customer to enter PIN.
- Customer enters PIN.
- ATM console verifies PIN with bank database and confirms the PIN is valid.
- If PIN is invalid, ATM console notifies the customer with a message and ejects the card.
- If PIN is valid, ATM console represents a prompt with options such as deposit, withdraw and transfer money.
- Customer selects withdraw option.
- ATM console presents prompt to the customer for an accurate amount to be withdrawn.
- The customer enters the amount.
- ATM console verifies with the bank database whether the customer account has sufficient funds. Otherwise, it shows an insufficient funds message.
- ATM console deducts money from customer's account, puts the requested amount in cash dispenser, and prints a receipt.
- ATM console rejects the card and becomes ready for another transaction.

Using the above flow of messages, construct the collaboration diagram similar to figure 1.38 where messages/method calls are flowing between objects with a sequence number. This sequence number specifies the ordering of the messages. The messages appear with pointing arrows from the sender object to the receiver object.

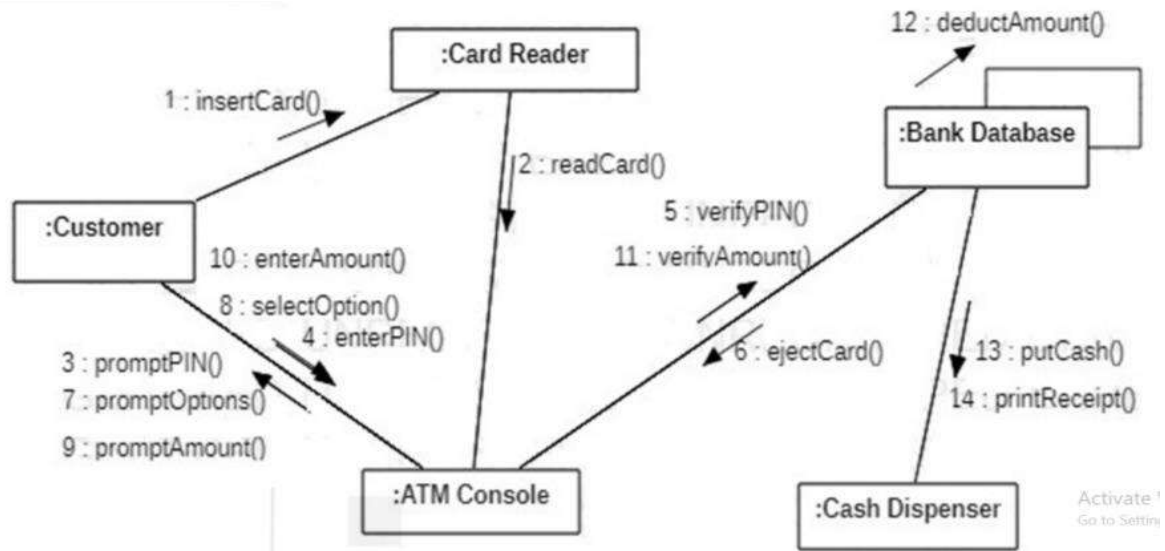


Figure 1.38: Collaboration Diagram for ATM transaction

When creating a collaboration diagram for the real business application, we can create methods as per need. The classes and their methods are created in the Collaboration Diagram given in figure 1.38, which can be summarized in the tabular form:

Table 13.1: Description of Classes and Methods

Class Name	Methods	Class Name	Methods
Customer	insertCard() enterAmount() selectOption() enterPIN()	Card Reader	readCard()
ATM Console	promptPIN() verifyPIN() promptOptions() promptAmount() verifyAmount()	Bank Database	ejectCard() deductAmount() putCash() printReceipt()

Now you can transform methods defined in the collaboration diagram into executable code. There are number of methods in the figure-1.38, but the pseudocode given below is written for verifyPIN() method only. This method verifies the customer's Personal Identification Number (PIN) using data retrieved from the ATM card's magnetic strip.

This method asks the customers to enter their PIN using the keypad of ATM console. If the PIN does not equal to the PIN stored on the card, then the ATM system allows a limited number of repeats. If it is still not matched, then the system invokes ejectCard() method and the card is seized as a security precaution. If the customer enters the correct PIN then the system invokes promptOptions() method where it provides options such as deposit money, withdraw cash and transfer money. You may try to write code for other methods as well.

The Pseudo Code for verifyPIN method as follows:

method verifyPIN

```
constants no_of_maxpins is 2
variable pin, pin_count is number
set pin_count to zero
read pin from ATMcard
loop until pin_count is equal to no_of_maxpins
    input pin from customer
    if entered pin matches with cardpin_database then
        call promptOptions method
    endif
    add 1 to pin_count
endloop
if pin_count is equal to no_of_maxpins then
    seize customer's card
else
    call ejectCard method
endif
```

endmethod// verifyPIN

The Java Code for verifyPIN method as follows:

```
public boolean verifyPIN(ATMCard ATMcard) {
    int constants no_of_maxpins = 2;
    int cardpin, pin_count, enteredpin;

    pin_count= 0;
    cardpin = ATMcard.readpin(); // pin stored in card
    while (pin_count == no_of_maxpins ){
        //pin entered by customer
        System.out.print("Enter pin: "); input(enteredpin);
        if (enteredpin == cardpin) then
            promptOptions();
        }
    }
```

```
        pin_count++;
    }//endloop
    if (pin_count == no_of_maxpins) {
        seizecustomercard();
    }
    else
        ejectCard();
    } //endif
} // endmethod of verifyPIN
```

6.3 Implementing the Collaboration Diagram Example 2

Consider another example of ‘**Programme Management System**’. Using this example, you can learn about the creation of methods from the collaboration diagram. This system contains **classes** like **Admin**, **ProgrammeCoordinator**, **Programme**, **Course** and **Faculty**.

- **Admin** initiates the manage programme functionality and interacts with Programme Coordinator.
- The **ProgrammeCoordinator** intermingled with different object classes in the system, such as Programme, Course and Faculty. The Programme Coordinator starts the process of creation and modification functionality of the programme.
- After the **programme** is either created or modified, ProgrammeCoordinator is responsible for the creation or modification functionality of a course.
- Each **programme** consists of many courses. It means that the class Programme and class Course are associated with each other in the form of one-to-many relationships.
- The **ProgrammeCoordinator** assigns the course(s) to faculty.
- Finally, the admin invokes the assigned faculty to the Course functionality, of the Programme.

The Collaboration Diagram for Programme Management System is shown in figure 1.39:

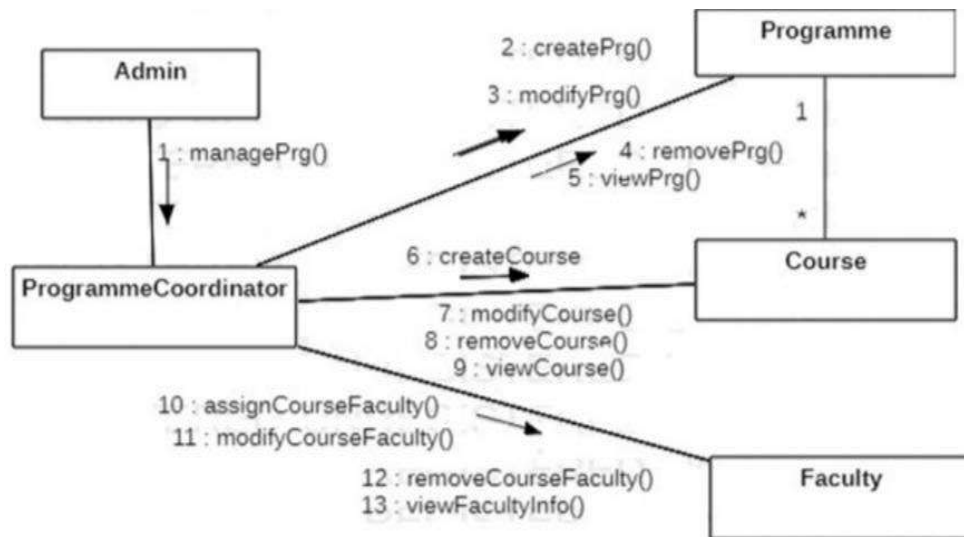


Figure 1.39: Collaboration Diagram for Programme Management System

The collaboration diagram for ‘Programme Management System’ shown in figure 1.39, involves the classes and contains the following methods. You may write code for the methods given in figure 1.39. List the basic assumptions, then try to write the code in the programming language you are comfortable.

Table 13.2: Description of Classes and Methods in Collaboration Diagram

Class Name	Methods	Class Name	Methods
Admin	managePrg()	Faculty	assignCourseFaculty() modifyCourseFaculty() removeCourseFaculty() viewFacultyInfo()
Programme	createPrg() modifyPrg() removePrg() viewPrg()	Course	createCourse() modifyCourse() removeCourse() viewCourse()

6.4 Implementing the Sequence Diagram

Consider the stock control program that stores details about orders which consist simply of a number of order lines, each of which specifies a certain number of parts of a particular type. Figure 1.40 shows a class diagram summarizing the necessary facts about orders. The relationship between orders and order lines is one of composition, because assume that when an order is destroyed all lines on that order are also destroyed.

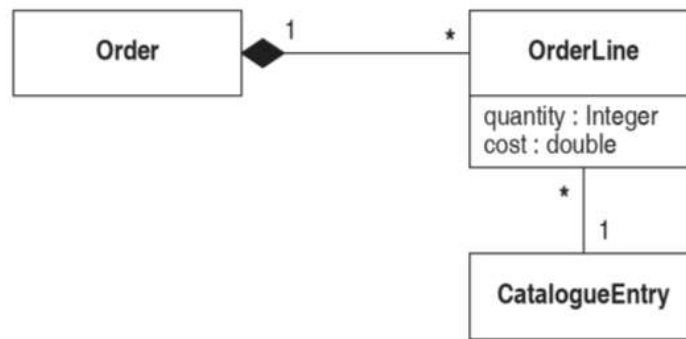


Figure 1.40: Orders in the stock control system

Let us assume that to create an order line, a client object must send a message to an order specifying the part and the quantity to be ordered. A new order line object will then be created and added to the order. A sequence diagram illustrating this interaction is shown in Figure 1.41.

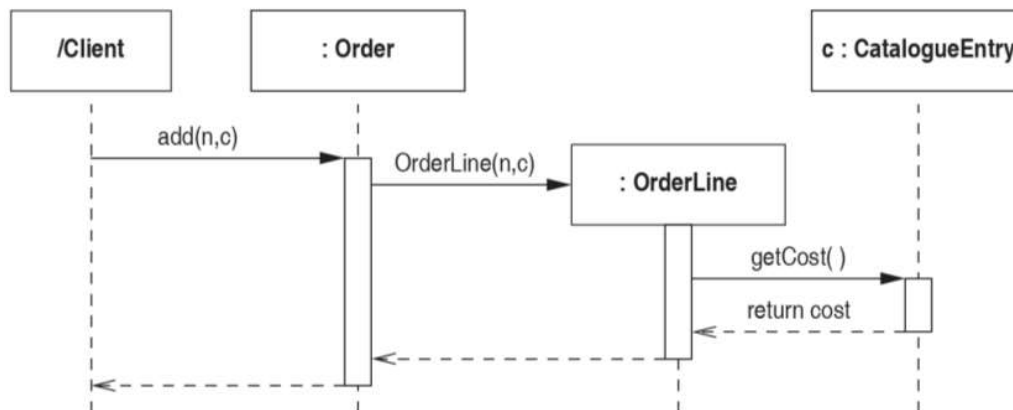


Figure 1.41: Sequence diagram for creating an order line

The new order line object is created in response to a message labelled ‘add’ sent from the client to the order. The parameters of this message supply the number of parts to be added and the catalogue entry object corresponding to the required parts. In response to this message, an order object creates a new order line object and the constructor call creating this object is shown as the next message in the interaction. The diagram shows that during the construction of order lines the cost of the relevant parts is retrieved from the catalogue entry object that was passed as a parameter. This value will be used in the initialization of the order line’s ‘cost’ attribute. At the end of this activation, control returns to the order and the order line’s lifeline continues as normal.

Collaboration diagrams cannot explicitly show the time at which a new object is created. In order to distinguish elements that are created during the course of an interaction from those that existed at the start, classifier and association roles

corresponding to new objects and links are annotated with the property ‘new’, shown in Figure 1.42.

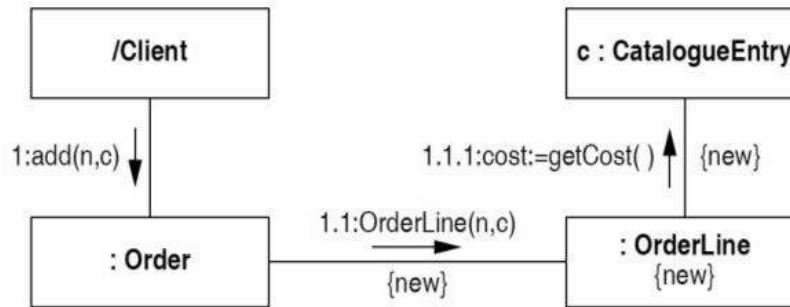


Figure 1.42: Object creation on a collaboration diagram

The classes and their methods are created in the Sequence Diagram given in figure 1.41, which can be summarized in the tabular form:

Table 13.1: Description of Classes and Methods

Class Name	Methods	Class Name	Methods
Order	OrderLine(int n, Catalogue c)	CatalogueEntry	
OrderLine	getCost()	ClientInterface	add(int n, Catalogue c)

Now transform interaction process defined in the Sequence Diagram into executable code. However only involved methods in the Sequence Diagram are defined as follows:

The Order.java, OrderLine.java, CatalogueEntry.java, Part.java

```

public class Order {

    private List<OrderLine> lineItems = new ArrayList<OrderLine>();

    .... //other mehods

    //create order line object and add into ArrayList
    public boolean OrderLine(int n, Catalogue c) {
        lineItems.add(new OrderLine(n, c));
        return true;
    }
    .... // other methods
    public void toString(){
        System.out.println( "Order includes: ");
        for(OrderLine item : lineItems){
            System.out.println( item.getCatName(); + " , " + item.getQty() + " , " +
  
```



```

        lineItem.getCost());
    }
}
public class OrderLine {

    private int qty;
    private CatalogueEntry catalog;
    private double cost;

    public OrderLine (int n, CatalogueEntry c) {
        this.qty = n;
        this.catalog = c;
        this.cost = c.getCost();
    }

    public String getCatName() { return catalog.getName(); }
    public int getQty() { return qty; }
    public double getCost() { return cost; }
}
public class CatalogueEntry
{
    private String name ;
    private long number ;
    private double cost ;

    public CatalogueEntry(String nm, long num, double cst) {
        name = nm ;
        number = num ;
        cost = cst ;
    }
    public String getName() { return name ; }
    public long getNumber() { return number ; }
    public double getCost() { return cost ; }
}

```

The test driver TestOrder.java

```

public class TestOrder {
    public static void main(String[] arg){
        CatalogueEntry screw = new CatalogueEntry("screw", 28834, 0.02) ;
        CatalogueEntry bolt = new CatalogueEntry("bolt", 10117, 0.03) ;
        CatalogueEntry hammer = new CatalogueEntry("hammer", 10223, 10.00) ;
        CatalogueEntry nail = new CatalogueEntry("nail", 10118, 0.04) ;

        Order order1 = new Order();
    }
}

```

```

order1.add(3, screw);
order1.add(4, bolt);
order1.hammer(1, hammer);
System.out.println(order1.toString());

Order order2 = new Order();
order2.add(20, nail);
order2.add(10, bolt);
System.out.println(order2.toString());
}

```

Example 2:

Consider the book issued by the librarian when customer requests to issue a book. The librarian first check the book is available. If yes, validate the member and checked the number of books issued by the member. If book can be issued, create issued transaction for the book and add member and book details. Then update the book status and member issued books status. The Collaboration diagram and Sequence diagram are shown below.

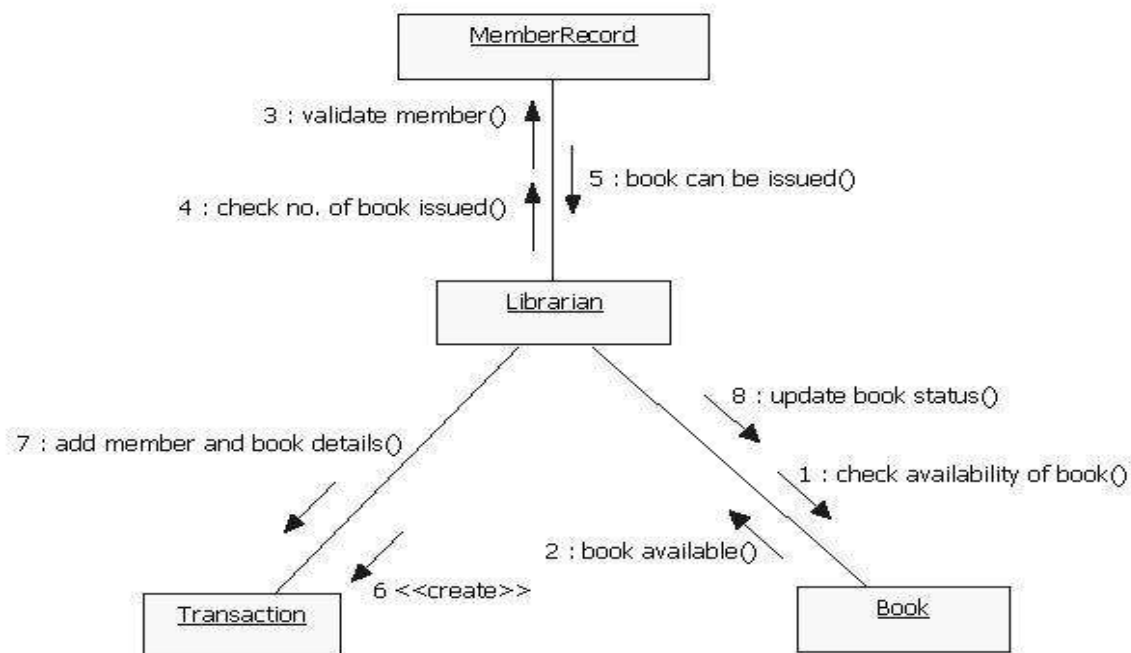


Figure 1.43: Collaboration diagram for issuing Book

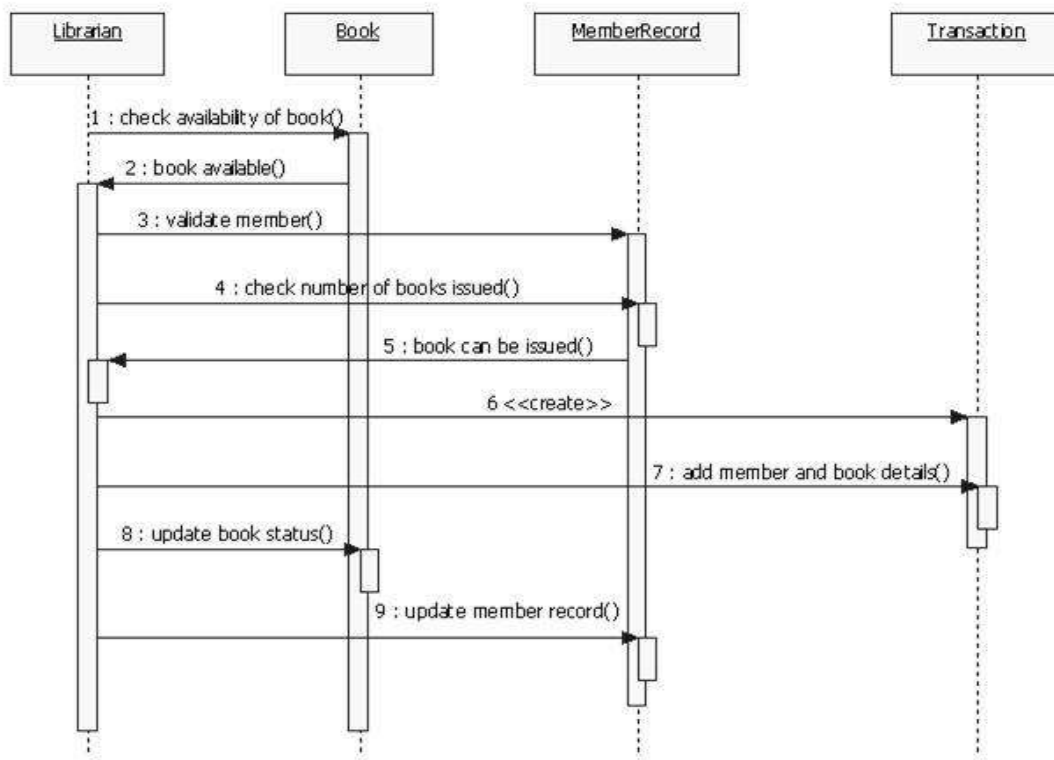


Figure 1.44: Sequence diagram for issuing Book

The classes and their methods are created in the Sequence Diagram given in figure 1.44, which can be summarized in the tabular form:

Table 13.1: Description of Classes and Methods

Class Name	Methods	Class Name	Methods
Librarian	checkAvailableofBook() updateBookStatus() validateMember() checknoofbooksIssued() addmemberandbookDetails() updateBookStatus() updatememberrecord()	MemberRecord	BookcanbeIssued()
Book	BookAvailable()	Transaction	

Now transform interaction process defined in the Sequence Diagram into executable code. However only involved methods in the Sequence Diagram are defined as follows:

The Librarian.java, Book.java, Transaction.java, MemeberRecord.java

```
public class Librarian {
```

```

BookCatalogue catalog = new BookCatalogue();
MemberList memberList = new MemberList();

..... //other mehods

public boolean IssuedBook(int bookID, int memberID) {
    //get book object from book catalogue List
    Book book = catalog.search(bookID);
    if (book.checkBookAvailable()) {
        System.out.println( "Book is available to issue." );
    } else {
        System.out.println( "Book is not available" );
        return false;
    }

    //get member object from memberList
    MemberRecord member = memberList.validateMember(memberID);
    if (member == null ) {
        System.out.println( "Invalid Member!" );
        return false;
    } else {
        System.out.println( "Member is valid." );
    }
    if (member.checkBookIssued()) {
        System.out.println( "Number of Books issued is under the Limit." );
    } else {
        System.out.println( "Number of Books issued is over the Limit." );
        return false;
    }
    // create issue book transaction
    Transaction trans = new Tranction( );
    trans.addIssuedBook(bookID(), memberID);
    System.out.println( "Book is issued successfully." );

    //update
    book.updateBookStatus(false);
    member.updateMemberRecord(bookID);
    return true;
}
..... // other methods
}

public class Book {

    private int bookID;
    private String bookTitle;

```

```

private String bookAuthor;
private String bookPublisher;
private boolean bookAvail = true;

..... // other methods
public boolean checkBookAvailable(){
    return this.bookAvail;
}
public void updateBookStatus(boolean status){
    this.bookAvail = status;
}
public int getID(){ return bookID; }
public String getTitle() { return bookTitle; }
public String getAuthor(){ return bookAuthor; }
private String getPublisher() { return bookPublisher; }

.... // other methods
}

```

```

public class MemberRecord {

    private int memberID;
    private String memName;
    private int bookLimit;
    private List<int> borrowedBooks = new ArrayList<int>();

    ... //other methods
    //Check the member ID is valid member or not
    public boolean validateMember(int mID){
        if (memberID == mID)
            return true;
        else
            return false;
    }
    //Check member borrowed books exceed the limit
    public boolean checkBookIssued( ){
        if (borrowedBooks.count() < bookLimit)
            return true;
        else
            return false;
    }
    //Record the borrowed book ID
    public void updateMemberRecord(int bookID){
        borrowedBooks.add(bookID);
    }
}

```

7. SOLID: Single, Open-Closed, Liskov Substitution and Dependency Inversion Principles

The SOLID principles are the fundamental principles that suggest for the object-oriented programming that -

- these principles are used to **determine how we should design classes**.
- these principles are used to **design software applications maintainable and testable**.

SOLID are principles, not patterns. SOLID is an acronym for the **five design principles** introduced by Robert C. Martin. SOLID stands for:

- (1) S = Single Responsibility Principle
- (2) O = Open/Closed Principle
- (3) L = Liskov Substitution Principle
- (4) I = Interface Segregation Principle
- (5) D = Dependency Inversion Principle

7.1 The Single Responsibility Principle (SRP)

The single responsibility principle states that:

“Each software module should have one and only one reason to change.”

In other words, **a class should have only one responsibility** and therefore it should have only one reason to change its code. If a class has more than one responsibility, then there will be more than one reason to change the class (code).

Now, the question is **what is responsibility?**

An application can have much functionality (features). For example, an online e-commerce application has many features such as displaying product lists, submitting an order, displaying product ratings, managing customers' shipping addresses, managing payments, etc. Along with these features, it also validates and persists products and customers' data, logs activities for auditing and security purposes, applies business rules, etc. You can think of these points as **functionalities or features or responsibilities**. **Change in any functionality leads to change in the class** that is responsible for that functionality.

Let's check how many responsibilities the following `Student` class has: